

Personalized Learning Path Generator for Course Platforms

A production-grade B2B SaaS project for developers targeting product-based companies

1. What Is This Product?

Businesses that run online courses - EdTech platforms, coaching institutes, corporate L&D teams, bootcamps - sell the same course to every student. A fresher and a 5-year experienced developer get the exact same syllabus, same module order, same content. That's the problem.

This SaaS platform lets businesses create and manage courses with modules, quizzes, and assessments. When a learner joins, they take a short onboarding assessment. The platform uses their responses to generate a fully personalized learning path - reordering modules, skipping content they already know, and surfacing the right material at the right time. AI generates a personalized welcome message and weekly progress summary email for each learner. The business sees completion rates, drop-off points, and learner health scores - all in one dashboard.

Why This Problem Is Real

- Online course completion rates globally sit between 5–15% - not because content is bad, but because it's not personalized
- Businesses lose revenue when learners abandon courses - refund requests, bad reviews, low renewal rates
- Existing tools (Teachable, Thinkific, Kajabi) have no intelligent path generation - every learner gets a cookie-cutter experience
- A business spending ₹50,000 on course creation loses all of it if learners drop off at module 2
- This is a problem every EdTech company, corporate training team, and coaching platform faces - it's not niche

Why AI Belongs Here - Naturally

Without AI: A course admin would need to manually create 10 different learning paths for 10 different learner profiles. At 100 learners, that's impossible.

With AI: The platform collects onboarding assessment data, feeds it to an LLM with the course structure as context, and gets back a personalized path + a personalized email - in seconds, for every single learner, automatically. AI is not the product. The course platform is. AI is what makes personalization scale.

2. Tech Stack Options

Learners can choose any stack they are comfortable with. The architecture and features remain identical across all options.

Layer	MERN Stack	Django + React	Spring Boot + React
Frontend	React.js + TailwindCSS	React.js + TailwindCSS	React.js + TailwindCSS
Backend	Node.js + Express.js	Django + DRF	Spring Boot (Java)
Database	MongoDB + Mongoose	PostgreSQL + psycopg2	PostgreSQL + JPA/Hibernate
Auth	JWT + bcrypt	JWT + djangorestframework-simplejwt	Spring Security + JWT
AI Integration	Anthropic / OpenAI SDK (Node)	Anthropic / OpenAI SDK (Python)	OpenAI Java SDK or REST call
Email	Nodemailer + SMTP	Django email backend + SMTP	JavaMailSender + SMTP
Job Scheduler	node-cron	Celery + Redis	Spring Scheduler
Deployment	Railway / Render + Vercel	Railway / Render + Vercel	Railway / Render + Vercel

Recommended for most learners: MERN (fastest to set up) or Django + React (cleanest ORM, easiest Celery integration for scheduled jobs). Spring Boot is the right pick if you are targeting Java-heavy companies like TCS, Infosys, or Zoho.

3. System Architecture

Core Modules

- **Module 1: Auth Service**
 - Multi-role: Admin (business), Learner
 - JWT-based auth with refresh token rotation
 - Role-based route guards on both frontend and backend

- **Module 2: Course Management Module**
 - Admin creates courses with ordered modules
 - Each module has: title, description, content URL/text, estimated duration, difficulty tag
 - Admin creates assessments: MCQ / short answer questions mapped to skill tags
 - CRUD APIs: courses, modules, assessments - all REST
- **Module 3: Learner Onboarding & Assessment Engine**
 - Learner joins a course and takes an onboarding assessment
 - Assessment scores are tagged by skill domain (e.g., Arrays: 80%, Trees: 20%)
 - Assessment result is stored per learner per course
- **Module 4: Personalized Path Generator**
 - After assessment, assessment result + full course module list is sent to LLM
 - LLM returns: reordered module sequence + modules to skip + reason for each decision
 - Path is stored in DB and rendered for the learner
 - Learner sees their custom path - not the generic course order
- **Module 5: Progress Tracking Engine**
 - Learner marks each module complete
 - Completion event is written to a progress table with a timestamp
 - Real-time progress bar updates on learner dashboard (polling or SSE)
 - Admin sees per-learner progress, drop-off module, and overall completion rate
- **Module 6: AI Email Generation Engine**
 - Trigger 1: After path generation → LLM generates personalized Day 1 welcome email (references learner name, their skill gaps, what they will work on first)
 - Trigger 2: Every Monday via cron job → LLM generates weekly progress summary email per learner (what they completed, what is pending, encouragement based on pace)
 - Emails are queued and sent via SMTP - logged in DB with status
- **Module 7: Admin Dashboard**
 - Course-level: total learners, avg completion %, drop-off module heatmap
 - Learner-level: individual progress, emails sent, last active timestamp
 - Learner health score = composite of completion pace + login recency + module scores

Data Flow Summary

- Learner signs up → takes onboarding assessment → scores computed
- Assessment result + course modules → LLM API call → personalized path returned
- Path stored in DB → learner dashboard renders personalized module order
- Learner completes modules → progress events written → real-time UI update
- Cron job fires every Monday → fetches all active learners → generates weekly email per learner via LLM → sends via SMTP
- Admin logs in → sees aggregated metrics across all learners and courses

4. MVP Feature List

Everything below should be built for the MVP. Optional features are clearly marked.

Feature	Category	Priority
Admin: Create / edit / delete courses	Course Management	Core MVP
Admin: Add modules to courses with metadata	Course Management	Core MVP
Admin: Create assessments with MCQ questions	Assessment Engine	Core MVP
Learner: Sign up and join a course	Auth & Onboarding	Core MVP
Learner: Take onboarding assessment	Assessment Engine	Core MVP
AI: Generate personalized learning path from assessment	AI Feature	Core MVP
Learner: View personalized module order on dashboard	Learner UX	Core MVP
Learner: Mark modules as complete	Progress Tracking	Core MVP
Real-time progress bar update on completion	Progress Tracking	Core MVP
AI: Generate personalized Day 1 welcome email	AI Feature	Core MVP
Cron: Weekly AI-generated progress email per learner	AI Feature	Core MVP
Admin: View course completion rate and drop-off module	Analytics	Core MVP
Admin: View per-learner progress and health score	Analytics	Core MVP
Learner health score computation	Analytics	Core MVP
Payment gateway (Stripe / Razorpay) for course access	Monetization	Optional
Admin: Export learner progress as CSV	Reporting	Optional
Learner: Re-take assessment and regenerate path	Advanced	Optional
Admin: A/B test two learning paths	Advanced	Optional

5. Step-by-Step Build Plan

Estimated timeline: 3–4 weeks solo. Each phase is shippable independently.

Phase 1 - Week 1: Foundation (Days 1–7)

- Set up project structure: monorepo or separate frontend/backend repos
- Configure database: schema design for users, roles, courses, modules, assessments, progress, emails
- Build Auth: register, login, JWT issue + refresh, role middleware
- Build Course CRUD APIs: create course, add modules, set module order
- Build Assessment CRUD APIs: create questions, map to skill tags

- Frontend: Auth pages (login, register), Admin dashboard shell, Course creation form

Phase 2 - Week 2: Core Engine (Days 8–14)

- Build Learner onboarding flow: join course → take assessment → submit
- Build assessment scoring engine: compute skill tag scores from answers
- Build LLM integration: send assessment result + module list → receive personalized path
- Store path in DB, render personalized module order on learner dashboard
- Build module completion API: mark complete, write progress event with timestamp
- Real-time progress bar: polling every 10s or SSE from backend

Phase 3 - Week 3: AI Email + Admin Views (Days 15–21)

- Build Day 1 welcome email: trigger after path generation, LLM generates personalized copy, send via SMTP
- Build cron job: every Monday, fetch all active learners, generate + send weekly summary email per learner
- Build email log table: track sent status, timestamp, learner ID
- Admin analytics view: course-level completion rate, drop-off module chart, learner list with health scores
- Learner health score formula: $(\text{completion \%} \times 0.5) + (\text{login recency score} \times 0.3) + (\text{avg module score} \times 0.2)$

Phase 4 - Week 4: Polish & Deploy (Days 22–28)

- Error handling: API error responses, LLM timeout fallback, email failure retry
- Input validation on all endpoints (Joi / Pydantic / Bean Validation)
- Rate limiting on AI endpoints to control LLM API costs
- Frontend polish: loading states, empty states, responsive layout
- Write a basic README with architecture diagram, setup steps, env vars
- Deploy backend to Railway or Render, frontend to Vercel
- Set up environment-based config: dev / production

6. Measurable Outcomes

Every metric below is computable from data the product already collects. No external analytics tool needed.

Metric 1	Course completion rate - % of learners who finish all modules in their personalized path
Metric 2	Drop-off module - the exact module where the most learners stop progressing

Metric 3	Avg time to path generation - how fast the LLM returns a path after assessment submission (in ms)
Metric 4	Email delivery rate - % of scheduled emails successfully sent vs failed
Metric 5	Learner health score distribution - how many learners are healthy vs at-risk vs inactive
Metric 6	Path personalization variance - avg difference in module order between learners (proves the path is actually different per person)

7. Portfolio Value & Skills Gained

What the Interviewer Will See

- A deployable, working SaaS - not a TODO app, not a weather app
- Multi-role system design with proper access control
- LLM integration that is purposeful - not AI for the sake of AI
- Event-driven design: completion events, cron-triggered jobs, email pipeline
- Measurable business metrics built into the product itself
- Clean REST API design they can actually explore

Skills the Learner Will Gain

- Database schema design for a real multi-entity SaaS product
- JWT auth with role-based access control - used in every product company
- LLM API integration: prompt engineering, structured output parsing, error handling
- Async job scheduling: cron jobs, background workers, email queues
- Real-time UI updates: SSE or polling pattern
- REST API design: versioning, status codes, error contracts
- Deployment: environment config, secrets management, CI-less deploy to cloud
- Business thinking: translating a user problem into measurable product metrics

How It Adds Value to Their Portfolio

- They can demo it live - a real URL, real login, real personalized path generation in front of the interviewer
- The GitHub repo shows architecture thinking - folder structure, separation of concerns, no spaghetti code

- They can explain every design decision: why cron and not event-based for weekly emails, why polling and not websockets for progress, why the health score formula works that way
- It is extendable - interviewers love asking 'how would you scale this?' - this project has clear, honest answers